



2. Tipos de datos y variables. Expresiones y operadores sobre tipos elementales.

Profesor: Juan Moreno García

Índice

1. Objetivos.
2. Tipos de datos.
3. Operadores.
4. Ejemplos.
5. Ejercicio.
6. Conclusiones.

1. Objetivos

1. Recordar los **tipos elementales de datos de Python**.
2. Aprender los **tipos elementales de datos de C**.
3. Aprender los **modificadores de C**.
4. Recordar los **operadores sobre tipos elementales de Python**.
5. Recordar el **ámbito de las variables en Python**.
6. Aprender el **ámbito de las variables en C**.
7. Recordar cómo **convertir entre tipos en Python**.
8. Aprender el **concepto de moldes (casts) en C**.

2. Tipos de datos.

Python	C
radio = 20	int radio; radio = 20;
pi = 3.141592	float pi; pi = 3.141592;
area = pi * radio**2	area = pi * pow(radio,2);

- * Ambos lenguajes distinguen entre **mayúsculas y minúsculas**.
- * Python asigna el **tipo de datos** mientras que **C hay que declararlo explícitamente**.
- * El **operador de asignación** es el mismo.
- * Las **órdenes de C deben terminar en ;**
- * **Python tiene más operadores que C**. Por ejemplo, el operador ****** no existe en C, que se puede realizar con la función **pow** (que también existe en Python).

2. Tipos de datos.

Tipos elementales de C

Tipos elementales de Python

- * int (entero)
- * float (real)
- * String (Cadena)

Tipo	Tamaño	Rango
void	0	0
boolean	1	true o false
char	1	-128 a 127
byte	1	0 a 255
int	2	-32.768 a 32.767
word	2	0 a 65.535
long	4	-2147483648 a 2147483647
float	4	-34028235E+38 a 34028235E+38
double	4	-34028235E+38 a 34028235E+38
string	Vector caracteres	Depende del numero de elementos
String	Objeto	Depende del numero de elementos

2. Tipos de datos.

* **C** dispone de modificadores como unsigned y const.

Tipo	Tamaño	Rango
unsigned char	1	0 a 255
unsigned int	2	0 a 65535
unsigned long	4	0 a 4294967295
const int pi = 3.1416;	4	-32.768 a 32.767

3. Operadores.

Op.	Descripción	Válido en
+	Suma	C y Python
-	Resta	C y Python
*	Multiplicación	C y Python
/	División	C y Python
%	Módulo (resto división entera)	C y Python
++	Incremento	C y Python
--	Decremento	C y Python
!	No lógico	C (not en Python)
&&	Y lógico	C (and en Python)
	O lógico	C (or en Python)
<<	Desplazamiento izquierda (binario)	C y Python
>>	Desplazamiento derecha (binario)	C y Python
**	potencia	Sólo en Python (en C función pow())

*** Ambos tienen prácticamente los mismos operadores sobre los tipos de datos básicos** excepto el ****** que sólo está en Python y los operadores lógicos.

3. Operadores.

Ámbito de variables

- * El ámbito de las variables se refiere a la **zona del programa donde una variable puede ser utilizada.**
- * Sólo se puede **utilizar donde sea declarada.**
- * Si se declara **dentro de una función sólo puede ser utilizada en esa función** y se denominan **variables locales.**
- * Este comportamiento es **igual para las funciones *setup()* y *loop()* de Arduino.**

3. Operadores.

```
1
2 def func(c):
3     b = 2
4     return c*b + a
5
6 def func1(c):
7     a = 1
8     a += 1
9     b = 2
10    return c*b + a
11
12 a = 4
13 r = 2
14
15 r1 = func(r)
16 print(a, r, r1)
17
18 r1 = func1(r)
19 print(a, r, r1)
```

```
int a;

int func(int c) {
    int b=2;
    return c*b + a;
}

int func1(int c) {
    int b=1, a=2;
    a +=1;
    return c*b + a;
}

void setup() {
    Serial.begin(115200); // Inicia puerto serie a 115200
    a = 4;
}

void loop() {
    int r = 2, r1;

    r1 = func(r);
    Serial.println( "\n" + String(a) + " " + String(r) + " " + String(r1) );

    r1 = func1(r);
    Serial.println( String(a) + " " + String(r) + " " + String(r1) );

    delay(5000);
}
```

3. Operadores.

Casts o Moldes

* El uso de moldes es necesario para realizar la conversión del tipo de una expresión al tipo deseado.

```
a = 1
b = float(a+1)
print('a =', a, '. b = ', b)

c = int(b)
print('b = ', b, '. c =', c)

d = a/b
e = int(a/b)
print('d = ', d, '. e =', e)
```

```
void loop() {
  int a, c;
  float b;
  float d, e;

  a = 1;
  b = (float)a+1;
  Serial.println( "a =" + String(a) + ". b =" + String(b) );

  c = (int) b;
  Serial.println("b =" + String(b) + ". c =" + String(c) );

  d = a/c;
  e = (float) a/c;
  Serial.println("d =" + String(d) + ". d =" + String(e) + '\n');

  delay(5000);
}
```

* En C se utilizan anteponiendo el tipo deseado a la expresión.

4. Ejemplos.

Programa que utiliza una variable global de tipo *int* que se incrementa en cada iteración. El programa principal muestra la suma de *inc* más el valor leído del Monitor Serie.

Versión Python

```
1 def incrementa(n):
2     n += inc
3     return n
4
5 inc = 0
6 n = 0
7 while n >= 0:
8     n = int( input('Dame un valor entero: ') )
9     r = incrementa(n)
10    print( 'inc = {}, n = {}. Resultado = {}'.format(inc, n, r) )
11    inc += 1
```

4. Ejemplos.

Programa que utiliza una variable global de tipo *int* que se incrementa en cada iteración de la función *loop()*. El *loop()* muestra la suma de *inc* más el valor leído del Monitor Serie.

Versión C

```
String leerLinea() {  
    // Espera hasta que haya algo que leer  
    while(Serial.available() < 1) {  
        delay(1);  
    }  
    // Recibe info hasta '\n'  
    return Serial.readStringUntil('\n');  
}  
  
int leerEntero() {  
    String r;  
    Serial.println("\n Dame un entero");  
    r = leerLinea(); // Lee entrada hasta '\n'  
    return r.toInt();  
}
```

```
int inc;  
  
int incrementa(int n) {  
    return n += inc;  
}  
  
void setup() {  
    Serial.begin(115200);  
    inc = 0;  
}  
  
void loop() {  
    int r, n;  
  
    // Lee el valor entero para sumarle inc  
    n = leerEntero();  
  
    // La variable es incrementada con inc  
    r = incrementa(n);  
  
    // Muestra el resultado de salida  
    Serial.println("inc = " + String(inc) + ", n = "  
        + String(n) + ". Resultado = " + String(r) );  
  
    // inc es incrementado para ver que se mantiene su valor  
    inc += 1;  
}
```

La variable *inc* debe ser declarada global para no perder su valor. Las variables locales del *loop()* se inicializan cada vez que éste se ejecuta.

5. Ejercicio.

Hacer un programa que dados dos enteros pedidos desde el Monitor Serie muestra la media, el producto y el resto de la división entera de dos valores enteros. El resultado deber ser un *float* en el caso de la media y entero para el resto. Se deben implementar una función para realizar cada uno de estos tres cálculos.

5. Conclusiones

1. Los tipos de datos elementales de Python están dentro de los de C. C tiene más tipos que Python.
2. **En C hay que declarar el tipo de datos de cada variable o parámetro de función.** Python los detecta automáticamente.
3. **El tamaño de las variables en Python es infinito**, mientras que **en C cada tipo de variable tiene un tamaño marcado por el sistema.**
4. **C dispone de modificadores** que permiten cambiar el tamaño de las variables y su rango de valores.
5. Los **operadores sobre tipos elementales** de C y Python son **prácticamente los mismos.**
6. **El ámbito de las variables puede ser global o local.** Las variables declaradas dentro de una función sólo se pueden utilizar en dicha función. Las variables declaradas fuera de toda función son globales y se pueden utilizar en cualquier lugar del mismo. **Esto es igual para Python y C.**

5. Conclusiones

7. Python tiene funciones específicas para convertir entre tipos, por ejemplo, `int()`, `float()`, etc.

8. C utiliza los casts o moldes para realizar la conversión de tipos elementales.